

Computer Organization Project 4 - Instruction Level Parallelism

Author: 劉庭瑋

Program 1: High ILP (low data dependencies)

```
#include <stdint.h>

volatile int result_sink;

int main(void) {
    register int a = 33;
    register int b = 66;
    register int c = 99;
    register int d = 132;
    register int s = 0;
    register int t = 0;
    register int u = 0;
    register int v = 0;
    register int w = 0;
    register int x = 0;
    register int y = 0;
    register int z = 0;
    register int r = 0;

    for (register int i = 1; i <= 100; i++) {
        s = a + b;
        t = c + d;
        u = a - c;
        v = b - d;
        w = a * b;
        x = c * d;
        y = a ^ d;
        z = b ^ c;
        r = a + b + c + d;
    }

    result_sink = s + t + u + v + w + x + y + z + r;

    return 0;
}
```

[Program 1 microarchitectural configuration 1] (limited superscalar)

```

16 def TAscalonXModel : SchedMachineModel {
15   let IssueWidth      = 2; // 2-way decode and dispatch
14   let MicroOpBufferSize = 32; // 32 micro-op re-order buffer
13   let LoadLatency     = 4; // Optimistic load latency
12   let MispredictPenalty = 14; // Fetch + Decode/Rename/Dispatch + Branch
11
10   let CompleteModel = false;
9
8   // TODO supported, but haven't added scheduling info yet.
7   let UnsupportedFeatures = [HasStdExtZbkb, HasStdExtZbkc, HasStdExtZbkx,
6                               HasStdExtZcmt, HasStdExtZknd, HasStdExtZkne,
5                               HasStdExtZknh, HasStdExtZksed, HasStdExtZksh,
4                               HasStdExtZkr];
3 }
2
1 let SchedModel = TAscalonXModel in {
114
1 //=====//
2 // Define each kind of processor resource and number available.
3
4 let BufferSize = 16 in {
5   def AscalonLS : ProcResource<3>;
6   def AscalonFXA : ProcResource<1>; // ALU, FP/VEC -> INT, MUL, DIV, CSR
7   def AscalonFXB : ProcResource<1>; // ALU, INT -> FP/VEC
8   def AscalonFXC : ProcResource<2>; // ALU, BR
9   def AscalonFXD : ProcResource<2>; // ALU
10  def AscalonFX : ProcResGroup<[AscalonFXA, AscalonFXB, AscalonFXC, AscalonFXD]>;
11  // FP
12  def AscalonFPA : ProcResource<1>; // Pipe A also handles FP/VEC -> INT
13  def AscalonFPB : ProcResource<1>;
14  def AscalonFP : ProcResGroup<[AscalonFPA, AscalonFPB]>;
15  // Vector
16  def AscalonVA : ProcResource<1>;
17  def AscalonVB : ProcResource<1>;
18  def AscalonV : ProcResGroup<[AscalonVA, AscalonVB]>;
19 }

```

[Program 1 microarchitectural configuration 2] (wider superscalar)

```

16 def TAscalonXModel : SchedMachineModel {
15   let IssueWidth      = 8; // 8-way decode and dispatch
14   let MicroOpBufferSize = 256; // 256 micro-op re-order buffer
13   let LoadLatency     = 4; // Optimistic load latency
12   let MispredictPenalty = 14; // Fetch + Decode/Rename/Dispatch + Branch
11
10   let CompleteModel = false;
9
8   // TODO supported, but haven't added scheduling info yet.
7   let UnsupportedFeatures = [HasStdExtZbkb, HasStdExtZbkc, HasStdExtZbkx,
6                               HasStdExtZcmt, HasStdExtZknd, HasStdExtZkne,
5                               HasStdExtZknh, HasStdExtZksed, HasStdExtZksh,
4                               HasStdExtZkr];
3 }
2
1 let SchedModel = TAscalonXModel in {
114
1 //=====//
2 // Define each kind of processor resource and number available.
3
4 let BufferSize = 16 in {
5   def AscalonLS : ProcResource<3>;
6   def AscalonFXA : ProcResource<1>; // ALU, FP/VEC -> INT, MUL, DIV, CSR
7   def AscalonFXB : ProcResource<1>; // ALU, INT -> FP/VEC
8   def AscalonFXC : ProcResource<2>; // ALU, BR
9   def AscalonFXD : ProcResource<2>; // ALU
10  def AscalonFX : ProcResGroup<[AscalonFXA, AscalonFXB, AscalonFXC, AscalonFXD]>;
11  // FP
12  def AscalonFPA : ProcResource<1>; // Pipe A also handles FP/VEC -> INT
13  def AscalonFPB : ProcResource<1>;
14  def AscalonFP : ProcResGroup<[AscalonFPA, AscalonFPB]>;
15  // Vector
16  def AscalonVA : ProcResource<1>;
17  def AscalonVB : ProcResource<1>;
18  def AscalonV : ProcResGroup<[AscalonVA, AscalonVB]>;
19 }

```

In-order issue (strictly speaking, llvm-mca models dispatch)

Look at the D positions. Earlier instructions get D before or at the same cycle as later instructions, indicating in-order dispatch.

[0,0]	DeER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	addi	sp, sp, -112
[0,1]	D=eER.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	ra, 104(sp)
[0,2]	D=eER.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s0, 96(sp)
[0,3]	D=eER.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s1, 88(sp)
[0,4]	D==eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s2, 80(sp)
[0,5]	D==eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s3, 72(sp)
[0,6]	D==eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s4, 64(sp)
[0,7]	D===eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s5, 56(sp)
[0,8]	.D==eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s6, 48(sp)
[0,9]	.D==eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s7, 40(sp)
[0,10]	.D===eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s8, 32(sp)
[0,11]	.D===eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s9, 24(sp)
[0,12]	.D===eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s10, 16(sp)
[0,13]	.D===eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s11, 8(sp)
[0,14]	.DeE----R	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	addi	s0, sp, 112
[0,15]	.DeE----R	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	li	s1, 33
[0,16]	.DeE----R	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	li	s2, 66
[0,17]	.DeE----R	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	li	s3, 99
[0,18]	.DeE----R	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	li	s4, 132
[0,19]	.DeE----R	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	li	a4, 0
[0,20]	.DeE----R	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	li	a3, 0

Out-of-order execution

[0,45] executes before [0,44], even though [0,44] appears earlier in program order, indicating an out-of-order execution. That means [0, 44] was waiting, in this case for the data dependency on a5.

[0,40]	.DeE-R.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sext.w	s11, a5
[0,41]	D=eeeER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	mulw	a5, s1, s2
[0,42]	D===eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sext.w	s10, a5
[0,43]	D=eeeER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	mulw	a5, s3, s4
[0,44]	D===eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sext.w	s9, a5
[0,45]	.DeE-----R	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	xor	a5, s1, s4
[0,46]	D=eE-----R	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sext.w	s8, a5
[0,47]	.DeE-----R	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	xor	a5, s2, s3

In-order commit (retirement)

Look at the relative positions of E and R. [0,14] finishes execution before [0,13], but it has to wait for [0,13] to retire.

[0,13]	.D===eER	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	sd	s11, 8(sp)
[0,14]	.DeE----R	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	.	addi	s0, sp, 112

Multiple instructions issued or committed in the same cycle

Multiple E(s) line up vertically in the timeline view -> multiple instructions issued in the same cycle.

Multiple R(s) line up vertically in the timeline view -> multiple instructions committed in the same cycle.

[illegible]

This is evident in high ILP programs because there are more independent instructions.

Performance comparison between the two configurations

There are no loop-carried data dependencies in this program, so we expect performance indicators to tend to their theoretical maximums.

The Performance Overview screenshot on the left is Configuration 1, and the one on the right is Configuration 2.

Iterations:	100	Iterations:	100
Instructions:	9700	Instructions:	9700
Total Cycles:	4855	Total Cycles:	1711
Total uOps:	9700	Total uOps:	9700
Dispatch Width:	2	Dispatch Width:	8
uOps Per Cycle:	2.00	uOps Per Cycle:	5.67
IPC:	2.00	IPC:	5.67
Block RThroughput:	48.5	Block RThroughput:	12.1

We can see that the program has real ILP, as moving “Dispatch Width” from 2 to 8 gives us a large performance gain, i.e., “IPC” from 2.00 to 5.67.

Interestingly, as “Dispatch Width” increases 4x, “Block RThroughput” also improves 4x, but “IPC” improves only ~2.84x. This is because there is still some level of dependency that limits scaling, i.e., the repeated sequence of addw / sext.w accumulating into a5 cannot be parallelized.

[2,30]	.	.	.	.	.	D=eE-----R	.	.	.	.	.	addw	a5, s1, s2
[2,31]	.	.	.	.	.	D==eE-----R	.	.	.	.	.	sxt.w	a5, a5
[2,32]	.	.	.	.	.	D===eE-----R	.	.	.	.	.	mv a4,	a5
[2,33]	.	.	.	.	.	D=eE-----R	.	.	.	.	.	addw	a5, s3, s4
[2,34]	.	.	.	.	.	.D=eE-----R	.	.	.	.	.	sxt.w	a5, a5
[2,35]	.	.	.	.	.	.D==eE-----R	.	.	.	.	.	mv a3,	a5
[2,36]	.	.	.	.	.	.DeE-----R	.	.	.	.	.	subw	a5, s1, s3
[2,37]	.	.	.	.	.	.D=eE-----R	.	.	.	.	.	sxt.w	a5, a5
[2,38]	.	.	.	.	.	.D==eE-----R	.	.	.	.	.	mv a2,	a5

In the Average Wait times table below, note the difference between [1] and [2]. The delta between the two counters is smaller for Configuration 1 (1st figure) because

performance is mostly limited by hardware resources. In contrast, the delta for Configuration 2 (2nd figure) is larger since performance is now limited by data dependencies and/or long latency instructions, according to the docs.

```
Average Wait times (based on the timeline view):
[0]: Executions
[1]: Average time spent waiting in a scheduler's queue
[2]: Average time spent waiting in a scheduler's queue while ready
[3]: Average time elapsed from WB until retire stage
```

[0]	[1]	[2]	[3]
-----	-----	-----	-----

10	1.9	0.5	0.5	<total>
----	-----	-----	-----	---------

10	4.7	0.9	5.2	<total>
----	-----	-----	-----	---------

In the Bottleneck Analysis view (enabled by the CLI flag *-bottleneck-analysis*) of Configuration 2, we can see several factors that limit scaling (from Configuration 1 to Configuration 2), which coincide with conclusions we drew above. Resource Pressure [81.30%] is the largest reported bottleneck, with the load/store unit (AscalonLS) being the biggest single resource class. Register Dependencies [58.21%] is also significant, caused by dependency chains such as *addw / sext.w*.

```
Cycles with backend pressure increase [ 87.32% ]
Throughput Bottlenecks:
  Resource Pressure      [ 81.30% ]
    - AscalonFXA [ 34.89% ]
    - AscalonFXB [ 34.72% ]
    - AscalonFXC [ 34.72% ]
    - AscalonFXD [ 34.72% ]
    - AscalonLS  [ 46.41% ]
  Data Dependencies:    [ 58.21% ]
    - Register Dependencies [ 58.21% ]
    - Memory Dependencies  [ 0.00% ]
```

Based on Configuration 2, if we increase AscalonLS from 3 to 6, we get substantial improvement in IPC/uOps Per Cycle. The relative pressure on the load/store unit decreases significantly.

```

Iterations:          100
Instructions:         9700
Total Cycles:        1613
Total uOps:          9700

Dispatch Width:      8
uOps Per Cycle:      6.01
IPC:                 6.01
Block RThroughput: 12.1

Cycles with backend pressure increase [ 86.30% ]
Throughput Bottlenecks:
  Resource Pressure   [ 55.36% ]
    - AscalonFXA [ 43.09% ]
    - AscalonFXB [ 30.87% ]
    - AscalonFXC [ 36.95% ]
    - AscalonFXD [ 30.87% ]
    - AscalonLS [ 12.28% ]
  Data Dependencies: [ 55.42% ]
    - Register Dependencies [ 55.42% ]
    - Memory Dependencies [ 0.00% ]

```

In the Resource Pressure view below, we can see that the pressure of Configuration 2 (second figure) is more uniformly distributed.

```

Resources:
[0] - AscalonFPA
[1] - AscalonFPB
[2] - AscalonFXA
[3] - AscalonFXB
[4.0] - AscalonFXC
[4.1] - AscalonFXC
[5.0] - AscalonFXD
[5.1] - AscalonFXD
[6.0] - AscalonLS
[6.1] - AscalonLS
[6.2] - AscalonLS
[7] - AscalonVA
[8] - AscalonVB

Resource pressure per iteration:
[0] [1] [2] [3] [4.0] [4.1] [5.0] [5.1] [6.0] [6.1] [6.2] [7] [8]
- - 9.00 8.00 10.75 10.75 15.75 15.75 9.00 9.00 9.00 - -

```

Resources:

- [0] - AscalonFPA
- [1] - AscalonFPB
- [2] - AscalonFXA
- [3] - AscalonFXB
- [4.0] - AscalonFXC
- [4.1] - AscalonFXC
- [5.0] - AscalonFXD
- [5.1] - AscalonFXD
- [6.0] - AscalonLS
- [6.1] - AscalonLS
- [6.2] - AscalonLS
- [7] - AscalonVA
- [8] - AscalonVB

Resource pressure per iteration:

[0]	[1]	[2]	[3]	[4.0]	[4.1]	[5.0]	[5.1]	[6.0]	[6.1]	[6.2]	[7]	[8]
-	-	10.99	10.99	11.99	12.00	12.01	12.02	9.00	9.00	9.00	-	-

Program 2: Low ILP (high data dependencies)

```
#include <stdint.h>
```

```
#include <stdio.h>
```

```
volatile uint64_t sink;
```

```
__attribute__((noinline)) uint64_t pipeline_bubbles_raw(uint64_t x) {
```

```
    x = x * 3 + 1;
```

```
    x = x * 3 + 1;
```

```
    x = x * 3 + 1;
```

```
    x = x * 3 + 1;
```

```
    x = x * 3 + 1;
```

```
    x = x * 3 + 1;
```

```
    x = x * 3 + 1;
```

```
    x = x * 3 + 1;
```

```
    return x;
```

```
}
```

```
__attribute__((noinline)) uint64_t register_renaming_demo(uint64_t a, uint64_t b,  
uint64_t c, uint64_t d) {
```

```
    uint64_t t0 = a + b;
```

```
    uint64_t t1 = c + d;
```

```
    uint64_t t2 = a * 3;
```

```
    uint64_t t3 = b * 5;
```

```
    uint64_t t4 = c * 7;
```

```
    uint64_t t5 = d * 9;
```

```
    return t0 + t1 + t2 + t3 + t4 + t5;
```

```
}
```

```

__attribute__((noinline)) uint64_t control_hazard_demo(uint64_t x, uint64_t n) {
    uint64_t sum = 0;
    for (uint64_t i = 0; i < n; i++) {
        if (x & 1) {
            sum += x;
        } else {
            sum -= x;
        }
        x = x * 1103515245 + 12345;
    }
    sink = sum;
    return sum;
}

int main(void) {
    uint64_t a = pipeline_bubbles_raw(10);
    uint64_t b = register_renaming_demo(5, 6, 7, 8);
    uint64_t c = control_hazard_demo(1, 5);

    sink = a + b;
    printf("%llu\n", (unsigned long long)sink);

    return 0;
}

```

[Program 2 microarchitectural configuration 1] (limited superscalar)

```

16 def TTAscalonXModel : SchedMachineModel {
~ 15   let IssueWidth      = 2; // 2-way decode and dispatch
14   let MicroOpBufferSize = 32; // 32 micro-op re-order buffer
~ 13   let LoadLatency    = 4; // Optimistic load latency
12   let MispredictPenalty = 14; // Fetch + Decode/Rename/Dispatch + Branch
11
10   let CompleteModel = false;
9
8   // TODO supported, but haven't added scheduling info yet.
7   let UnsupportedFeatures = [HasStdExtZbkb, HasStdExtZbkc, HasStdExtZbkx,
6                               HasStdExtZcmt, HasStdExtZknd, HasStdExtZkne,
5                               HasStdExtZknh, HasStdExtZksed, HasStdExtZksh,
4                               HasStdExtZkrr];
3 }
2
1 let SchedModel = TTAscalonXModel in {
114
1 //=====//
2 // Define each kind of processor resource and number available.
3
4 let BufferSize = 16 in {
5   def AscalonLS : ProcResource<3>;
6   def AscalonFXA : ProcResource<1>; // ALU, FP/VEC -> INT, MUL, DIV, CSR
7   def AscalonFXB : ProcResource<1>; // ALU, INT -> FP/VEC
8   def AscalonFXC : ProcResource<2>; // ALU, BR
9   def AscalonFXD : ProcResource<2>; // ALU
10  def AscalonFX : ProcResGroup<[AscalonFXA, AscalonFXB, AscalonFXC, AscalonFXD]>;
11  // FP
12  def AscalonFPA : ProcResource<1>; // Pipe A also handles FP/VEC -> INT
13  def AscalonFPB : ProcResource<1>;
14  def AscalonFP : ProcResGroup<[AscalonFPA, AscalonFPB]>;
15  // Vector
16  def AscalonVA : ProcResource<1>;
17  def AscalonVB : ProcResource<1>;
18  def AscalonV : ProcResGroup<[AscalonVA, AscalonVB]>;
19 }

```

[Program 2 microarchitectural configuration 2] (wider superscalar)

```

12 def TTAscalonXModel : SchedMachineModel {
13   let IssueWidth          = 8; // 8-way decode and dispatch
14   let MicroOpBufferSize = 256; // 256 micro-op re-order buffer
13   let LoadLatency        = 4; // Optimistic load latency
12   let MispredictPenalty = 14; // Fetch + Decode/Rename/Dispatch + Branch
11
10   let CompleteModel = false;
9
8   // TODO supported, but haven't added scheduling info yet.
7   let UnsupportedFeatures = [HasStdExtZbkb, HasStdExtZbkc, HasStdExtZbkd,
6                               HasStdExtZcmt, HasStdExtZknd, HasStdExtZkne,
5                               HasStdExtZknh, HasStdExtZksed, HasStdExtZksh,
4                               HasStdExtZkrj];
3 }
2
1 let SchedModel = TTAscalonXModel in {
114 //=====
1 // Define each kind of processor resource and number available.
3
4 let BufferSize = 16 in {
5   def AscalonLS : ProcResource<3>;
6   def AscalonFXA : ProcResource<1>; // ALU, FP/VEC -> INT, MUL, DIV, CSR
7   def AscalonFXB : ProcResource<1>; // ALU, INT -> FP/VEC
8   def AscalonFXC : ProcResource<2>; // ALU, BR
9   def AscalonFXD : ProcResource<2>; // ALU
10  def AscalonFX : ProcResGroup<[AscalonFXA, AscalonFXB, AscalonFXC, AscalonFXD]>;
11  // FP
12  def AscalonFPA : ProcResource<1>; // Pipe A also handles FP/VEC -> INT
13  def AscalonFPB : ProcResource<1>;
14  def AscalonFP : ProcResGroup<[AscalonFPA, AscalonFPB]>;
15  // Vector
16  def AscalonVA : ProcResource<1>;
17  def AscalonVB : ProcResource<1>;
18  def AscalonV : ProcResGroup<[AscalonVA, AscalonVB]>;
19 }

```

Pipeline bubbles caused by data hazards, control hazards, or resource stalls, etc.

Instructions 24-28 show a serial dependency chain through register a5. The waiting cycles are caused by RAW data hazards. For example, instruction 25 cannot execute before instruction 24 produces the new value of a5. Note that while they are waiting, other independent instructions can still execute in the same cycle.

```
[0,24] . D====eE---R. . . . . . . . . . . . . . . . mv a5, a4
[0,25] . D=====eE---R. . . . . . . . . . . . . . . . slli a5, a5, 1
[0,26] . D=====eE---R. . . . . . . . . . . . . . . . add a5, a5, a4
[0,27] . D=====eE-R. . . . . . . . . . . . . . . . addi a5, a5, 1
[0,28] . D=====eER. . . . . . . . . . . . . . . . sd a5, -24(s0)
[0,29] . DeeeeE-----R. . . . . . . . . . . . . . . . ld a4, -24(s0)
```

Instruction 75 provides clear evidence of a resource stall caused by load/store unit pressure. We can see that instruction 62, which produces s0, finishes execution one cycle before instruction 75 starts executing. LLVM-MCA marks this instruction with "RESOURCE interference: AscalonLS [probability: 100%]", meaning that the simulator attributes its delay to contention for the load/store execution resource.

The screenshot below is generated by the *-bottleneck-analysis* CLI option.

Evidence of register renaming eliminating WAW or WAR hazards

[illegible]

Performance comparison between the two configurations

The Performance Overview screenshot on the left is Configuration 1, and the one on the right is Configuration 2. We can see that increasing “Dispatch Width” from 2 to 8 improves this low ILP program, reducing “Total Cycles” from 18413 to 6344 and increasing “IPC” from 1.03 to 2.98. This shows that the program still has some level of exploitable parallelism.

Iterations:	100	Iterations:	100
Instructions:	18900	Instructions:	18900
Total Cycles:	18413	Total Cycles:	6344
Total uOps:	19300	Total uOps:	19300
Dispatch Width:	2	Dispatch Width:	8
uOps Per Cycle:	1.05	uOps Per Cycle:	3.04
IPC:	1.03	IPC:	2.98
Block RThroughput:	96.5	Block RThroughput:	28.3

In the bottleneck analysis view below, configuration 1 is mostly limited by frontend (or Dispatch Width). As the backend is underfed, LLVM-MCA does not observe significant functional-unit pressure. Configuration 2 exposes RAW dependencies (Register Dependencies [44.92%]) and load/store unit contention (AscalonLS [27.54%]).

```
Cycles with backend pressure increase [ 13.58% ]
Throughput Bottlenecks:
  Resource Pressure      [ 0.00% ]
  Data Dependencies:    [ 13.58% ]
    - Register Dependencies [ 13.58% ]
    - Memory Dependencies [ 0.00% ]
```

```
Cycles with backend pressure increase [ 46.50% ]
Throughput Bottlenecks:
  Resource Pressure      [ 29.11% ]
    - AscalonFXA [ 3.12% ]
    - AscalonFXB [ 3.12% ]
    - AscalonFXC [ 4.70% ]
    - AscalonFXD [ 3.12% ]
    - AscalonLS [ 27.54% ]
  Data Dependencies:    [ 44.92% ]
    - Register Dependencies [ 44.92% ]
    - Memory Dependencies [ 0.00% ]
```

Based on Configuration 2, if we increase AscalonLS from 3 to 6, the Resource Pressure decreases significantly, a lot more than the decrease in Program 1. Block RThroughput also decreases given the additional hardware resource. However, IPC / uOPs Per Cycle does not seem to improve as much, mainly because the program is still heavily limited by instruction dependencies.

```

Iterations:          100
Instructions:         18900
Total Cycles:         6194
Total uOps:           19300

Dispatch Width:      8
uOps Per Cycle:       3.12
IPC:                  3.05
Block RThroughput:    24.1

Cycles with backend pressure increase [ 37.16% ]
Throughput Bottlenecks:
  Resource Pressure   [ 9.65% ]
    - AscalonFXA [ 3.21% ]
    - AscalonFXB [ 3.21% ]
    - AscalonFXC [ 4.83% ]
    - AscalonFXD [ 3.21% ]
    - AscalonLS  [ 4.83% ]
  Data Dependencies:  [ 35.57% ]
    - Register Dependencies [ 35.57% ]
    - Memory Dependencies  [ 0.00% ]

```

Comparison between Program 1 & 2

1. Performance

IPC and uOps per cycle for Program 1 are both significantly larger than those of Program 2. With Dispatch Width 2, the observed uOps Per Cycle of Program 1 reaches its theoretical maximum, (uOps per iteration / Block Throughput) = 2, which means it fully saturates the narrow issue width, whereas Program 2 only reaches 1.05.

Moving from Dispatch Width 2 to 8, Program 1 scales strongly, reaching IPC/uOps Per Cycle of 5.67. In contrast, Program 2 only reaches IPC 2.98.

These results show that superscalar processors provide the largest benefit when the program exposes enough instruction-level parallelism.

2. Pipeline behaviors

In Program 1, the timeline view shows multiple instructions dispatched, executed, and retired in the same cycle, with fewer long scheduler waits. In

Program 2, the timeline view shows more '=' wait cycles caused by RAW dependencies, plus load/store unit pressure under the wider configuration. Therefore, high-ILP code keeps the superscalar pipeline busy, while low-ILP code creates pipeline bubbles because dependent instructions must wait for earlier results.

Microarchitectural configurations such as larger Issue Width and increased Functional Unit Counts (e.g., FXA, FXB, FP, Load/Store units) are most effective when the program exposes enough independent instructions for the hardware scheduler to exploit. As for Functional Unit Counts, we can examine the bottleneck analysis result to see where hardware resources may be added for the largest performance boost.